Props with function

- Props stands for properties.
- Props are like function arguments, and you send them into the component as attributes.

```
function Person(props) {
            return <h3>Hello Person of Age: {props.age}!</h3>;
}
```

```
const root = createRoot(document.getElementById('root'));
root.render(<Person age="23"/>);
```

Note: props can be changed to any name.

Props with function

```
function Person(props) {
            return <h3>
            Hello Person {props.age}    {props.text.name}
            </h3>;
}
```

```
const root = createRoot(document.getElementById('root'));
root.render(<Person age="23" text={{name: 'ram'}} />);
```

Props with class

```
import React from "react";

class Person extends React.Component {
        render() {
        return <h3>
        Hello Person {this.props.age}   {this.props.text.name}
        </h3>;
        }
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Person age="23" text={{name: 'ram'}} />);
```

Class vs Function Component

Class Components	Function Components
Class components need to extend the component from “React.Component” and create a render function that returns the required element.	Functional components are like normal functions which take “props” as the argument and return the required element.
They are also known as stateful components.	They are also known as stateless components.
They implement logic and the state of the component.	They accept some kind of data and display it in the UI.
Lifecycle methods can be used inside them.	Lifecycle methods cannot be used inside them.
Format of Hooks inside class components: `constructor(props) { super(props); this.state = { color: ‘ ‘ } }`	Format of Hooks inside function components: `const [color,SetColor]= React.useState('')`
It needs to store state therefore constructors are used.	Constructors are not used in it.
It has to have a “render()” method inside that.	It does not require a render method.

States

State refers to a built-in object that stores **data** or **values** that can **change over time**, and when it changes, the component **re-renders** to reflect the new values.

Imagine you're building:

- A **counter** that increases on a button click
- A **form** that stores user input
- A **dark/light theme** toggle

In all of these, **values change dynamically**, and you need React to **respond to that**. That's where state comes in.

Syntax:

```
const [stateVariable, setStateFunction] = useState(initialValue);
```

Example 1: Counter

```
import React, { useState } from 'react';

function Counter() {
	const [count, setCount] = useState(0);

	return (
		<div>
		<h2>Count: {count}</h2>
		<button onClick={() => setCount(count + 1)}>Increment</button>
		</div>
	);
}
```

What's Happening:

- useState(0) sets count to 0 initially.
- Clicking the button runs setCount(count + 1).
- React re-renders the component with the new count.

Example 2: Form Input

```
import React, { useState } from 'react';

function Form() {
	const [name, setName] = useState(‘ ’);

	return (
		<div>
			<input type="text” value={name} onChange={(e) =>
			setName(e.target.value) }
			/>
			<p>Hello, {name}</p>
		</div>
	);
}
```

States

State is local and private

Each component manages its **own state**. One component's state does **not affect others**, unless you **explicitly pass** data via props or global state management.

Rules of useState

- Only call useState at the top level of the component
- Don't call it inside loops or conditions
- Always use the setter function (setStateFunction) to update the value

States vs Props

Feature	state	props
Data source	Internal (local to component)	External (passed from parent)
Editable	Yes	No (read-only)
Used for	Interactivity, dynamic UI	Passing data to child components

Use of ... (Spread Operator)

The **...** spread operator is used to **copy all the values** of an object or array into a new object/array.

When updating state that is an **object or array**, we should **never mutate the original state directly**.
React relies on **immutability** to detect changes and re-render the UI.
So instead of changing the original object, we create a **new copy** with updated values — and this is where ... is useful.

Example: Updating Object State

```
const [user, setUser] = useState({ name: "Rahul", age: 25 });

const updateAge = () => {
setUser({ ... user, age: 26 }); // spread previous user, change only age
};
```

Explanation:

- ... user spreads all existing key-value pairs (name: "Rahul")
- Then age: 26 overrides the old age
- You're creating a new object, not mutating the old one

Example: Updating Array State

```
const [items, setItems] = useState(["Java", "Python"]);

const addItem = () => {
setItems([... items, "React"]); // spreads old items, adds new one
};
```

Explanation:

- ... items creates a copy of the existing array
- "React" is added to the end
- No mutation — safe for React to detect changes